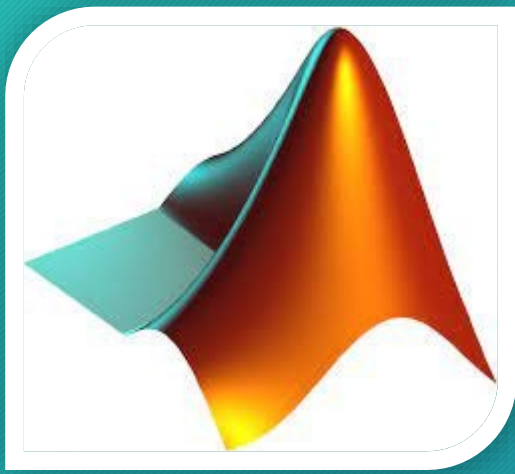




Communication Design

LAB.5

Wireless Communications
Pulse-Shaping Filters

Ass.Lec.Ammar Alabbassi

Course Outline:

✓ 1- Simulate a Basic Digital Communications Link

- ▶ Implement the essential components of a single-carrier digital communications system, including additive white Gaussian noise.

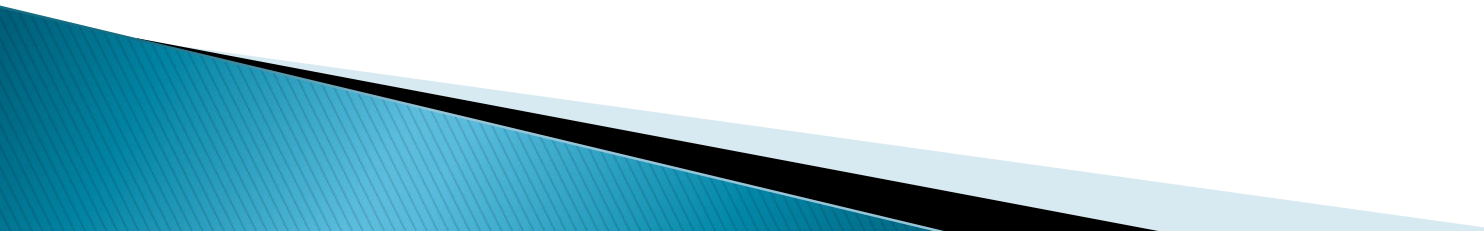
✓ 2- Pulse-Shaping Filters

- ▶ Incorporate transmit and receive filters into your simulation.

3- Multipath Channels

- ▶ Model a multipath channel and evaluate the impact on link quality.

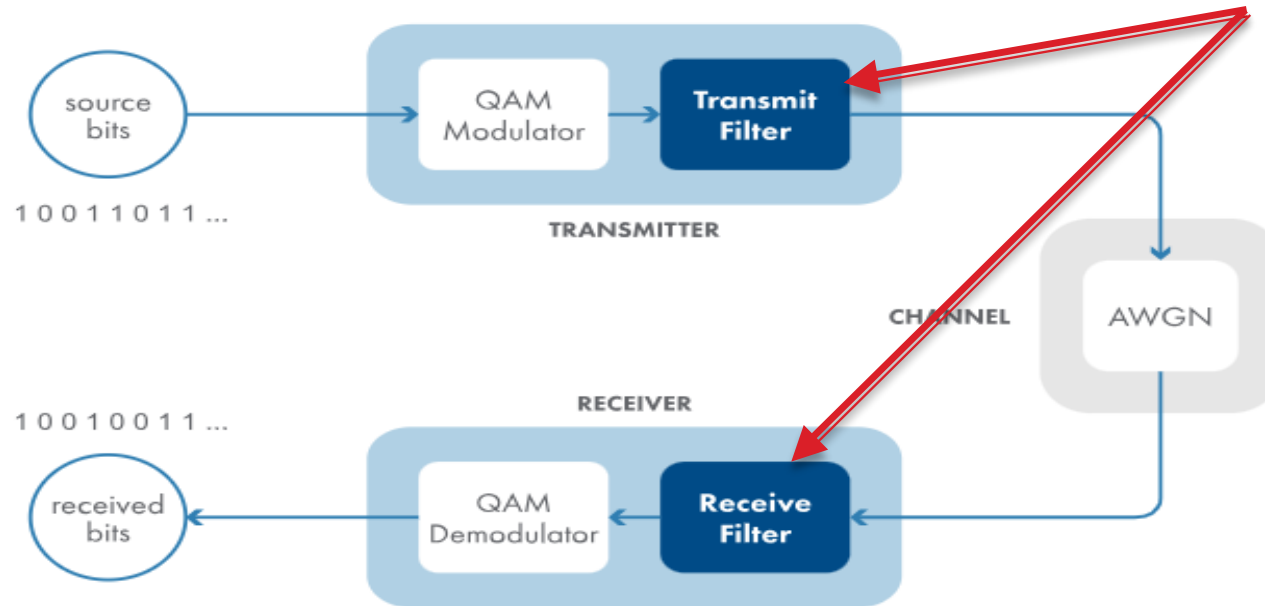
4- OFDM

- ▶ Implement orthogonal frequency-division multiplexing (OFDM), a multicarrier modulation scheme.
- 

2- Pulse-Shaping Filters

Transmit and Receive Filters

In this activity, you'll add square-root raised cosine filters to the transmitter and receiver. The transmit filter takes the 16-QAM signal as input, and its output is sent over the channel. The receive filter takes the channel output as its input, and its output is passed to the QAM demodulator.



These filters help in constraining the occupied bandwidth of the waveform without introducing intersymbol interference (ISI).

2- Pulse-Shaping Filters

Transmit and Receive Filters

Task 1

This code sets up a 16-QAM link.

% Simulation parameters

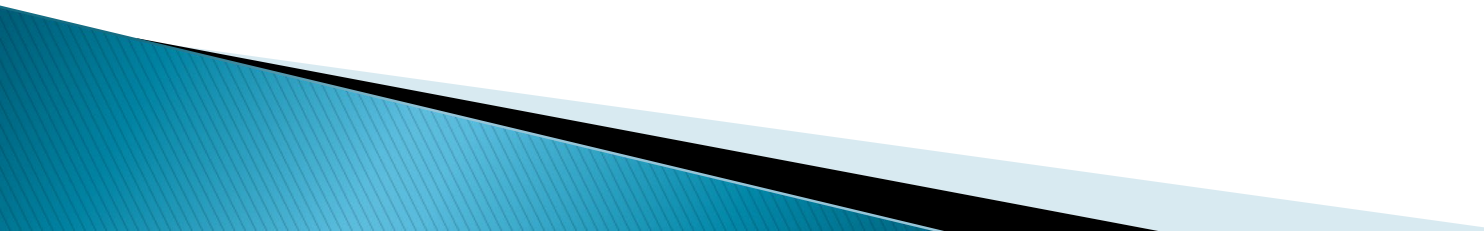
```
numBits = 20000;
```

```
modOrder = 16;
```

% Create source bit sequence and modulate using 16-QAM.

```
srcBits = randi([0,1],numBits,1);
```

```
modOut = qammod(srcBits,modOrder,"InputType","bit","UnitAveragePower",true);
```



Task 1

Background

- Because systems typically use matched filters, it's common to create the transmit and receive filters at the same time.
- You can create a square-root raised cosine transmit filter using the following syntax.

```
txFilt = comm.RaisedCosineTransmitFilter
```

- A similar syntax creates the matched receive filter.

```
rxFilt = comm.RaisedCosineReceiveFilter
```

- These syntaxes set the filter properties to default values. If you leave off the semicolon, you can see some of the properties in the output panel. For example, the property **OutputSamplesPerSymbol** has the default value 8, which means the filter **upsamples** the signal so it has 8 samples for each QAM symbol.

TASK

Create square-root raised cosine transmit and receive filters. Name the transmit filter **txFilt** and the receive filter **rxFilt**.

To display the filter properties, you can leave off the semicolons at the end of the commands.

```
txFilt = comm.RaisedCosineTransmitFilter  
rxFilt = comm.RaisedCosineReceiveFilter
```

```
txFilt =  
comm.RaisedCosineTransmitFilter with properties:  
    Shape: 'Square root'  
    RolloffFactor: 0.2000  
    FilterSpanInSymbols: 10  
    OutputSamplesPerSymbol: 8  
    Gain: 1  
  
rxFilt =  
comm.RaisedCosineReceiveFilter with properties:  
    Shape: 'Square root'  
    RolloffFactor: 0.2000  
    FilterSpanInSymbols: 10  
    InputSamplesPerSymbol: 8  
    DecimationFactor: 8  
    DecimationOffset: 0  
    Gain: 1
```

Task 2

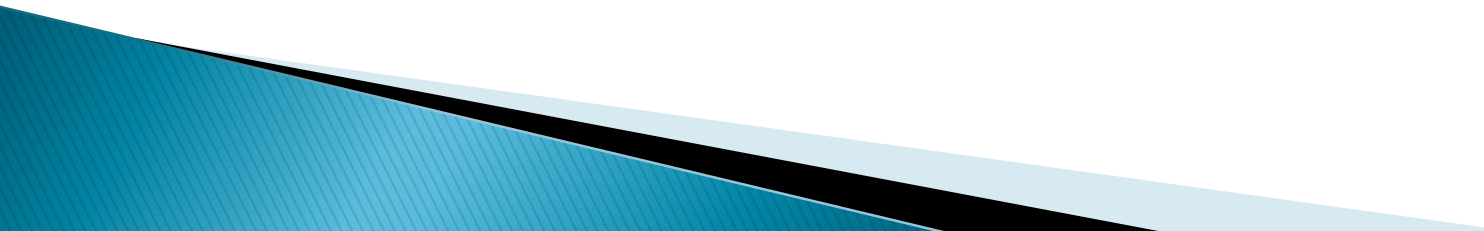
- To apply the transmit filter to the modulated 16-QAM signal, pass the signal as an input to the stored filter.

```
sigOut = txFilt(sig)
```

TASK

- Apply the transmit filter `txFilt` to the modulated 16-QAM signal `modOut`. Name the output signal `txFiltOut`.

```
txFiltOut = txFilt(modOut);
```



Task 3

- Notice that (**txFiltOut**= 40000) is 8 times longer than (**modOut**=5000), which is expected given the value of the filter property **OutputSamplesPerSymbol**.
 - Next, you want to apply **AWGN** to the filtered signal. However, the transmit filter changed the signal power.
 - To scale the noise power appropriately, pass a third input to the **awgn** function.
-
- **sigOut = awgn(sig,SNR,"measured")**
 - When you use the "measured" option, the function calculates the input signal power and scales the noise power based on the signal-to-noise ratio.

TASK

Apply **AWGN** to the filtered signal **txFiltOut**. Use a signal-to-noise ratio of **7 dB**. Name the output **chanOut**.

SNR = 7;

chanOut = awgn(txFiltOut,SNR,"measured");

Task 4

- Now apply the receive filter to the signal. As with the transmit filter, pass the signal as an input to the stored filter.

```
sigOut = rxFilt(sig)
```

TASK

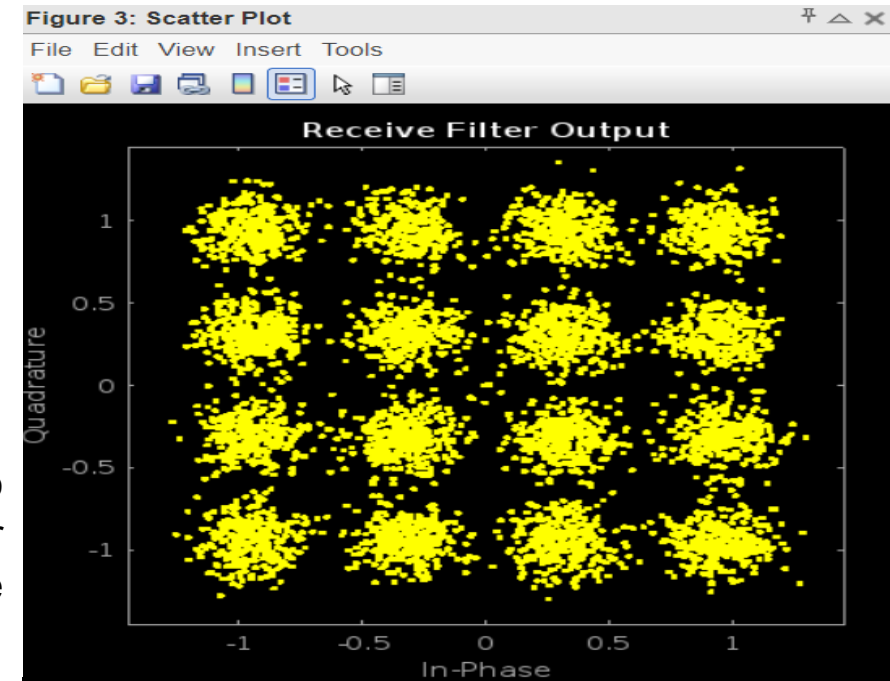
- Apply the receive filter `rxFilt` to the channel output `chanOut`. Name the output `rxFiltOut`.

```
rxFiltOut = rxFilt(chanOut);
```

Demodulate back into bits. This code only executes after `rxFiltOut` has been created.

```
if exist("rxFiltOut","var") % code runs after you complete Task 4
scatterplot(rxFiltOut)
title("Receive Filter Output")
demodOut = qamdemod(rxFiltOut,modOrder,
"OutputType","bit","UnitAveragePower",true);
end
```

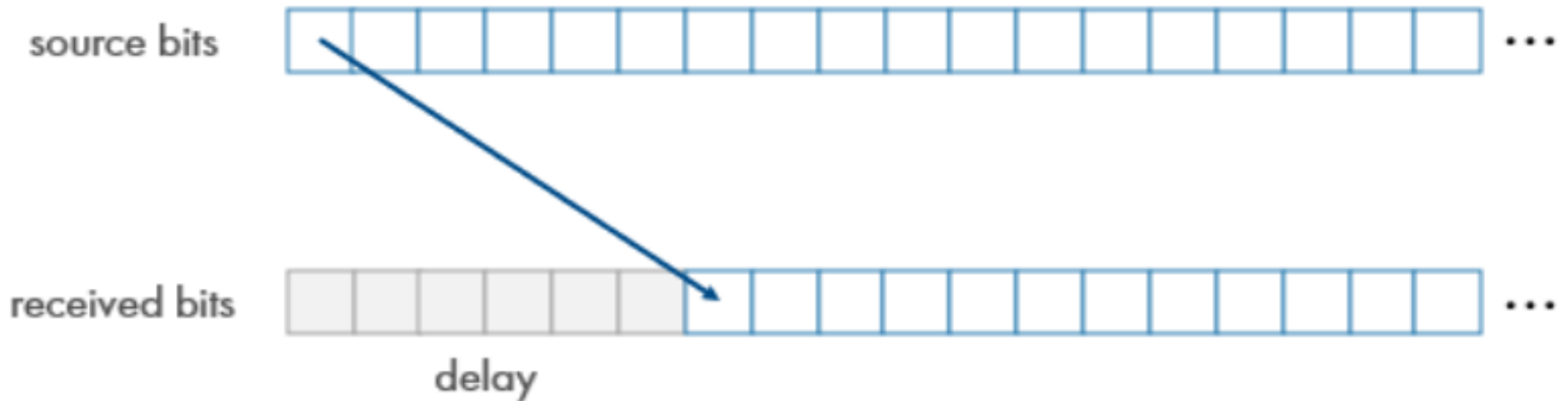
Remember you added more noise to the channel here (SNR of 7 dB) compared to the previous activity (SNR of 15 dB). The receive filter has reduced the noise power by about 8 dB, so the constellation has roughly the same amount of noise as the signal from the previous activity.



2- Pulse-Shaping Filters

Filter Delay

Transmit and receive filters incur delays on the signal, which must be accounted for when the two bit sequences are compared.



2- Pulse-Shaping Filters

Filter Delay and BER Calculation

Task 1

This code simulates a 16-QAM link with filtering and AWGN.

```
numBits = 20000;
modOrder = 16; % for 16-QAM
bitsPerSymbol = log2(modOrder) % modOrder = 2^bitsPerSymbol
txFilt = comm.RaisedCosineTransmitFilter;
rxFilt = comm.RaisedCosineReceiveFilter;

srcBits = randi([0,1],numBits,1);
modOut = qammod(srcBits,modOrder,"InputType","bit","UnitAveragePower",true);
txFiltOut = txFilt(modOut);

SNR = 7; % dB
chanOut = awgn(txFiltOut,SNR,"measured");

rxFiltOut = rxFilt(chanOut);
demodOut = qamdemod(rxFiltOut,modOrder,"OutputType","bit","UnitAveragePower",true);
```

Task 1

Background

- For each filter, the delay is half the filter length, in symbols.
- You can extract the filter length by accessing the **FilterSpanInSymbols** property. Then divide by 2 to get the delay.

`txFilt.FilterSpanInSymbols/2`

`rxFilt.FilterSpanInSymbols/2`

The total delay is the sum of the transmit and receive filter delays.

TASK

Calculate the total delay (in symbols) by summing the transmit filter delay and receive filter delay. Name the result **delayInSymbols**.

`delayInSymbols = rxFilt.FilterSpanInSymbols/2 + txFilt.FilterSpanInSymbols/2`

`delayInSymbols = 10`

Task 2

- However, to compare bits, you need to determine the delay in **bits**, not **symbols**.
- For 16-QAM, four bits are mapped to one symbol. This conversion factor has been stored in the variable **bitsPerSymbol**, defined in the script.
- To convert the delay from **symbols** to **bits**, multiply the delay in symbols by **bitsPerSymbol**.

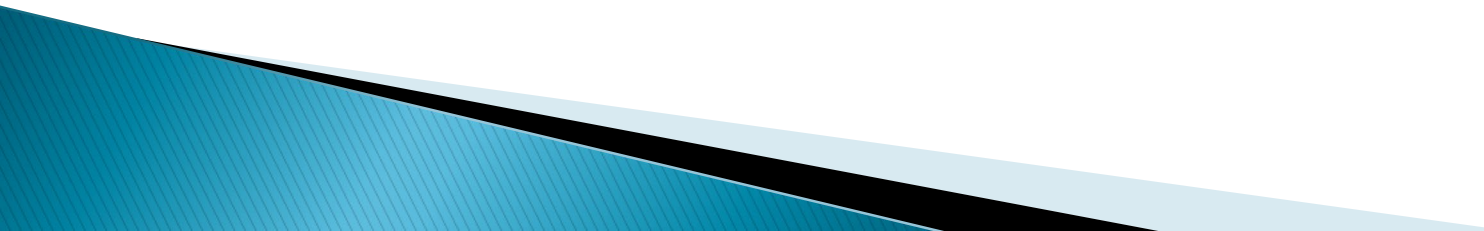
`delayInSymbols * bitsPerSymbol`

TASK

- Calculate the delay in bits by converting **delayInSymbols** to **bits**. Name the result **delayInBits**.

`delayInBits = delayInSymbols * bitsPerSymbol`

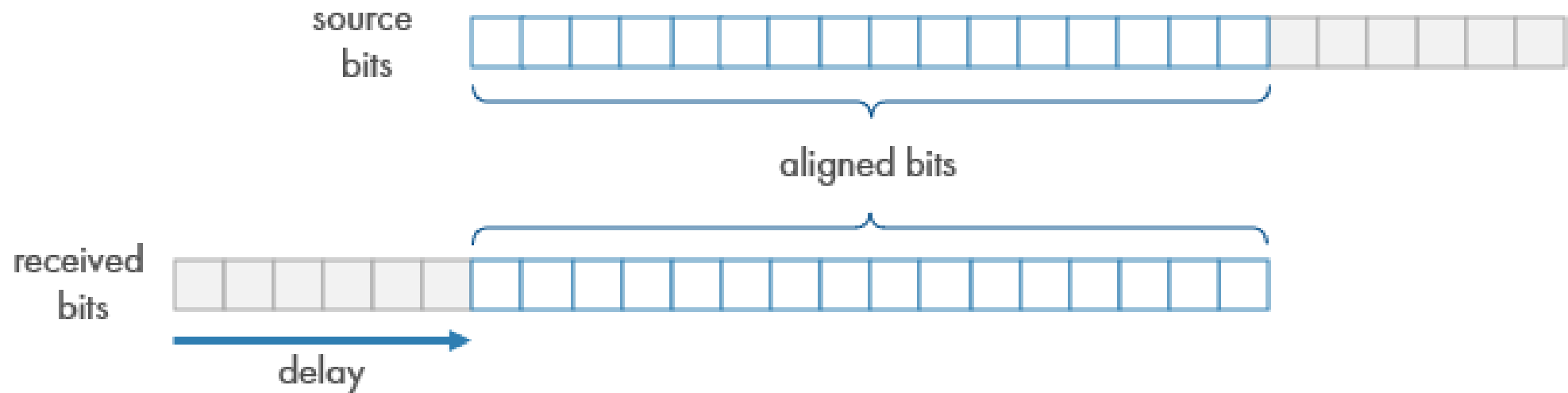
`delayInBits = 40`



Task 3

Extract Aligned Bits

Now that you know the delay, you're ready to compare bits. To compare the correct bits, you need to align matching bits from the source bit sequence and the received bit sequence.



Task 3

Background



- First, get the aligned bits from the source bit sequence by extracting all but the last `delayInBits` bits.
`srcBits(1:(end-delayInBits))`

TASK

- Extract the aligned source bits from `srcBits`, and store them in the variable `srcAligned`.
`srcAligned = srcBits(1:(end-delayInBits))`

```
bitsPerSymbol = 4
delayInSymbols = 10
delayInBits = 40
srcAligned = 19960x1
```

	1
1	0
2	0
3	0
4	0
5	1
6	0
7	1
8	1



- Next, get the aligned bits from the received bit sequence by extracting all but the first `delayInBits` bits.
`demodOut((delayInBits+1):end)`

TASK

- Extract the aligned received bits from `demodOut`, and store them in the variable `demodAligned`.
`demodAligned = demodOut((delayInBits+1):end)`

demodAligned = 19960x1

	1
1	0
2	0
3	0
4	0
5	1
6	0
7	1
8	1

Task 5

- Now you can compare the **aligned bits** and count the **number of bit errors**.
- Remember you can use the `~=` operator to perform a bit-by-bit comparison to find nonmatching bits, and then use `nnz` to count the bit errors.

```
nnz(bitSeq1 ~= bitSeq2)
```

TASK

- Count the number of bit errors between **srcAligned** and **demodAligned**. Store the result in the variable **numBitErrors**.

```
numBitErrors = nnz(srcAligned ~= demodAligned)
```

```
numBitErrors = 44
```

Task 6

- The aligned bit stream is shorter than the original.

`numAlignedBits = length(srcAligned)`

- To calculate the bit error rate, you need to divide the number of bit errors by the number of aligned bits.

TASK

- Calculate the bit error rate, and store it in the variable **BER**.

`numAlignedBits = length(srcAligned);`

`BER = numBitErrors/numAlignedBits`

BER = 0.0022